Justification des choix

Groupe 3 — Projet B (emplois du temps), approche agentique

Version de travail — novembre 2026

Chaque choix indique les alternatives possibles, la raison retenue et sa limite. La colonne « Décidé par » distingue ce qui a été fixé par les 4 groupes (socle commun), par notre groupe, ou reste à trancher.

1 Technologies

Choix	Alternatives possibles	Pourquoi	Limite	Décidé par
Python + FastAPI	Django, Node/ Express, Spring	Langage maîtrisé par toute la promo (DE et DFS) : pas de temps d'apprentissage, et un retard ne pourra pas être attribué à la techno. Spécification OpenAPI générée depuis le code, qui sert de contrat d'API. Même langage pour l'API et les algorithmes.	Python est lent pour le calcul intensif : à surveiller pour le recuit simulé (benchmark).	4 groupes
PostgreSQL 16	MySQL, MongoDB	Données très relationnelles (séances, profs, groupes, salles). Clés étrangères et contraintes `CHECK` pour la cohérence ; transactions pour vérifier les conflits et écrire une séance d'un seul bloc.	—	4 groupes
JWT + rôles (RBAC)	sessions côté serveur	Front et API séparés, chaque groupe ayant son propre front : un jeton porté par chaque requête évite de partager un état de session. Rôles stockés en base (`ADMIN`, `MANAGER`, `TEACHER`, `STUDENT`).	Révocation d'un jeton plus difficile : durée courte et jeton de rafraîchissement.	4 groupes
Docker compose	installation locale	Même environnement pour les 5 membres et pour le serveur ; une commande pour tout lancer.	Conflits de ports avec d'autres projets locaux (réglables par `.env`).	4 groupes
SQLAlchemy 2 + Alembic	SQL brut (psycopg)	Le schéma évolue sans recréer la base : chaque changement est une migration versionnée et réversible. Un test vérifie que modèles et migrations ne divergent pas.	Couche d'abstraction à apprendre ; quand utiliser du SQL brut reste à trancher (issue #9).	groupe 3
pytest sur le vrai PostgreSQL	base simulée (mocks), SQLite	Teste exactement ce qui tourne en production (contraintes,	Tests un peu plus lents et dépendants du compose.	groupe 3

		transactions) ; chaque test est annulé à la fin, donc indépendant.		
React/Next.js + Tailwind + shadcn/ui	Vue, Angular	*À justifier par les DFS.*	—	à trancher

2 Architecture du code

Choix	**Pourquoi**	**Limite**
Backend découpé par couche : routes, accès aux données, modèles, schémas	Chaque fichier a une seule responsabilité : on teste l'accès aux données sans HTTP, et l'agent applique la même structure partout grâce à un exemple de référence (`GET /roles`).	Plus de fichiers pour une petite route.
Génération dans un module Python pur (`scheduling/`)	Aucun lien avec la base ni avec l'API : les algorithmes se testent et se comparent sur des fichiers d'instances, sans rien démarrer.	Une passerelle base ↔ modèle reste à écrire.
Un évaluateur unique pour tous les algorithmes	Même juge, mêmes instances, mêmes graines aléatoires : la comparaison montrée au prof est équitable.	Le résultat dépend des poids choisis pour le score souple.
Contrôles de sécurité systématiques	Requêtes paramétrées, entrées validées par Pydantic, mots de passe en bcrypt, aucun secret dans le code : règles écrites dans `AGENTS.md`, donc appliquées aussi par l'agent.	Repose sur la relecture des PR tant que la CI n'existe pas.

3 Algorithmes de génération

Choix	**Pourquoi**	**Limite**
Heuristiques plutôt qu'une méthode exacte	Le problème est NP-difficile (il contient la coloration de graphe) : l'optimum est hors d'atteinte sur de grandes instances.	Aucune garantie d'optimalité.
Chaîne glouton → coloration (DSatur) → recuit simulé	Le glouton sert de base de comparaison ; la coloration modélise directement les conflits et donne une solution valide ; le recuit améliore ensuite les contraintes souples sans casser les dures.	Le recuit demande des réglages (température, refroidissement) et n'est pas déterministe.
Pas de solveur (OR-Tools) pour l'instant	Garder des algorithmes que l'on conçoit et explique soi-même, comme le demande le cadrage.	Pas de référence optimale pour mesurer l'écart de nos heuristiques.
Pas d'appel à un LLM pour générer les emplois du temps	Exigence du cadrage : la partie algorithmique doit rester la nôtre.	—
Grille au pas de 30 min, cours de 1 h ou 1 h 30, 8 h 30–12 h et 13 h 30–18 h 30	Correspond au fonctionnement réel des cours ; un cours ne déborde jamais sur la pause de midi.	Durées hors de ces valeurs non gérées.

Génération semaine par semaine	Chaque semaine peut différer (jours fériés, indisponibilités) ; un problème par semaine reste de taille raisonnable.	Avec les mêmes règles, les semaines risquent de se ressembler.
Trous pénalisés en priorité dans le score	C'est le critère de confort le plus visible pour les étudiants ; un trou de moins d'une heure coûte moins, mais pas rien.	Les poids sont un choix humain, à ajuster d'après le benchmark.

4 Organisation et outillage

Choix	**Pourquoi**	**Limite**
Branches `feature/*` → `dev` → `main`, tout par PR	Relecture humaine de chaque changement, `main` toujours stable ; les PR fournissent les métriques de processus (délai de revue, taux d'acceptation).	Des push directs ont eu lieu : protéger `main` et `dev` sur GitHub.
Contexte agent versionné (`AGENTS.md`)	Même contexte et mêmes règles pour chaque session. Les dépôts qui versionnent la configuration de leur agent dégradent moins leur qualité [1].	Fichier à tenir à jour ; tout le monde n'utilise pas un agent qui le lit.
Commencer avec un agent seul, sans sous-agents	Recommandation de partir simple et de n'ajouter de la complexité que si elle apporte un gain mesurable [2].	Le gain des sous-agents reste à mesurer.
Journal en CSV + trailers git	Une ligne par tâche, analysable par script ; l'origine du code (IA ou humaine) est lue dans git sans saisie, et les données servent directement au rapport.	Dépend de la discipline de chacun ; pas de tokens pour les outils autres que Claude Code.
VPS de l'équipe + GitHub Actions + ghcr	Même compose qu'en local, pas de mise en veille comme sur les hébergeurs gratuits, et un serveur déjà disponible.	Maintenance et sauvegardes à notre charge ; serveur en ARM, image à construire pour cette architecture.
Rapports en Typst	Texte versionné dans git comme le code, compilation instantanée, bibliographie gérée.	Outil peu connu de l'équipe.

Références

[1] Y. Denisov-Blanch *et al.*, « A Few Pages of Markdown: Committed AI Configuration and Lower Quality Cost after Coding-Agent Adoption ». [En ligne]. Disponible sur: https://arxiv.org/abs/2608.25241

[2] E. Schluntz et B. Zhang, « Building effective agents ». [En ligne]. Disponible sur: https://www.anthropic.com/engineering/building-effective-agents